CENG7880
Trustworthy and Responsible AI

Instructor: Sinan Kalkan

(https://ceng.metu.edu.tr/~skalkan)

For course logistics and materials:
https://metu-trai.github.io

MIDDLE EAST TECHNICAL UNIVERSITY
DEPARTMENT OF COMPUTER ENGINEERING

Previously on CENG7880!

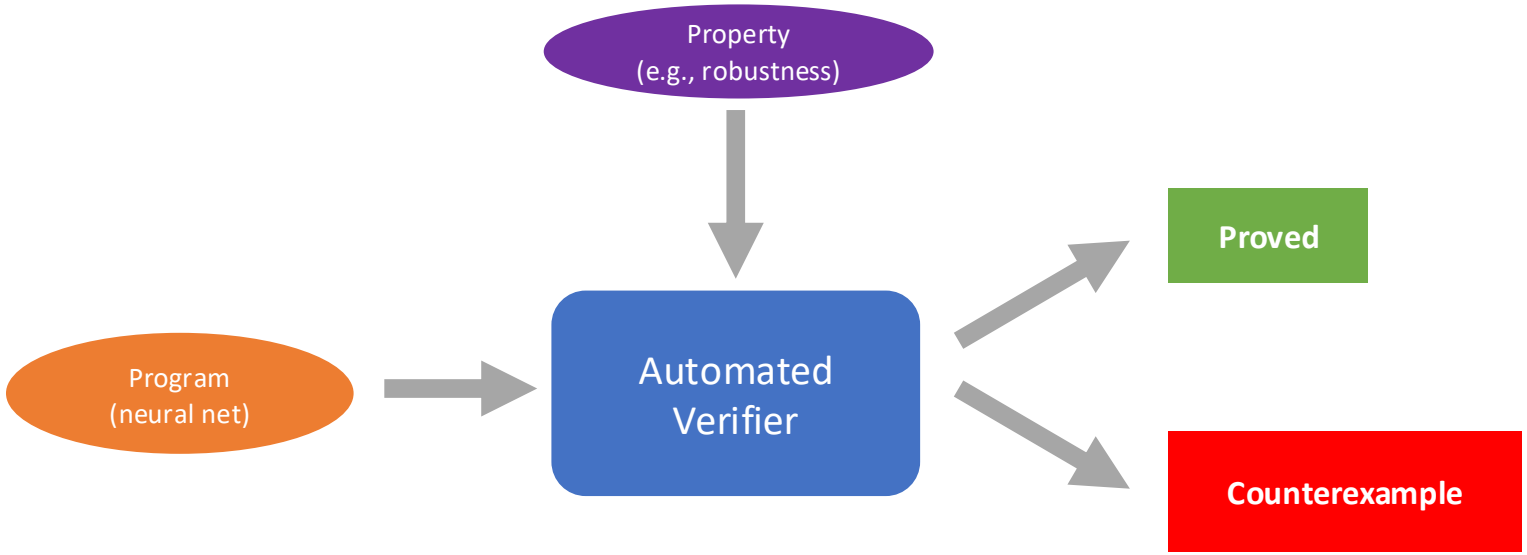
Adversarial Defense Mechanisms

- Option 1: Detect attacks before feeding them to the model
  - Consider perturbed images as out-of-distribution
  - E.g., train an encoder-decoder network on “clean” input and use reconstruction error as a sign of ”anomaly”.

- Option 2: Purify perturbed inputs
  - Remove perturbations/attacks
  - Transform perturbed input to its closest “clean” version
  - E.g., use denoising autoencoders

Wang, Y., Sun, T., Li, S., Yuan, X., Ni, W., Hossain, E., & Poor, H. V. (2023). Adversarial Attacks and Defenses in Machine Learning-Powered Networks: A Contemporary Survey. arXiv preprint arXiv:2303.06302.

Previously on CENG7880!

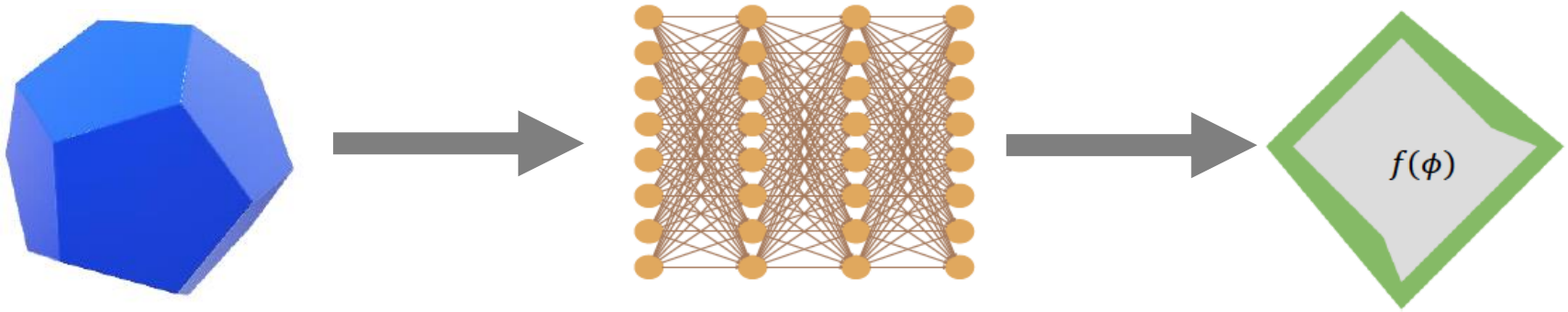


The general problem is computationally intractable, therefore we may need to provide an approximate answer

- **Acknowledgement for slides:**
  - **CIS 6730 (Computer Aided Verification) Lectures**
  - **Formal methods in AI: Gagandeep Singh and Madhu Parthasarathy (UIUC)**

Previously on CENG7880!

Neural network certification: problem statement



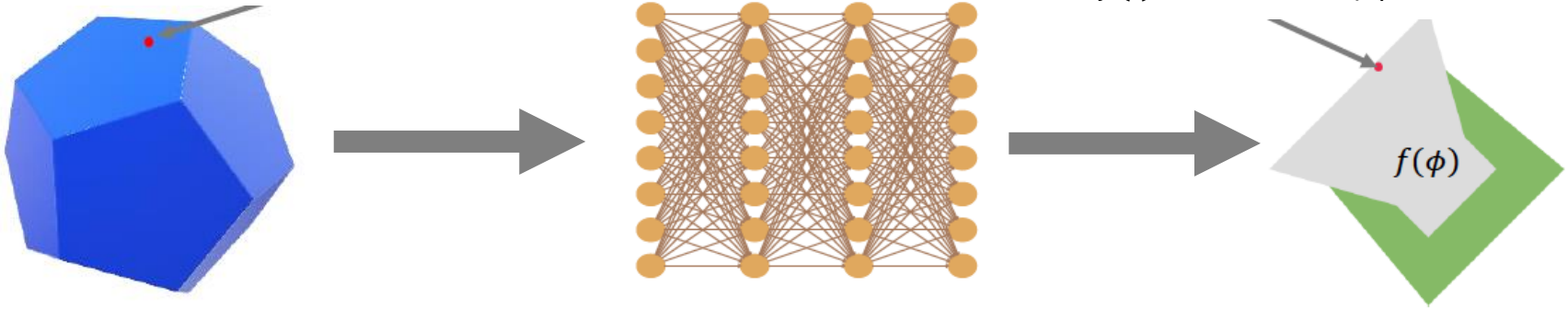
Precondition over network inputs ϕ

Network f

Prove $f(\phi) \subseteq \psi$

Find a counter example $i \in \phi$

$f(i)$ does not satisfy ψ



Precondition over network inputs ϕ

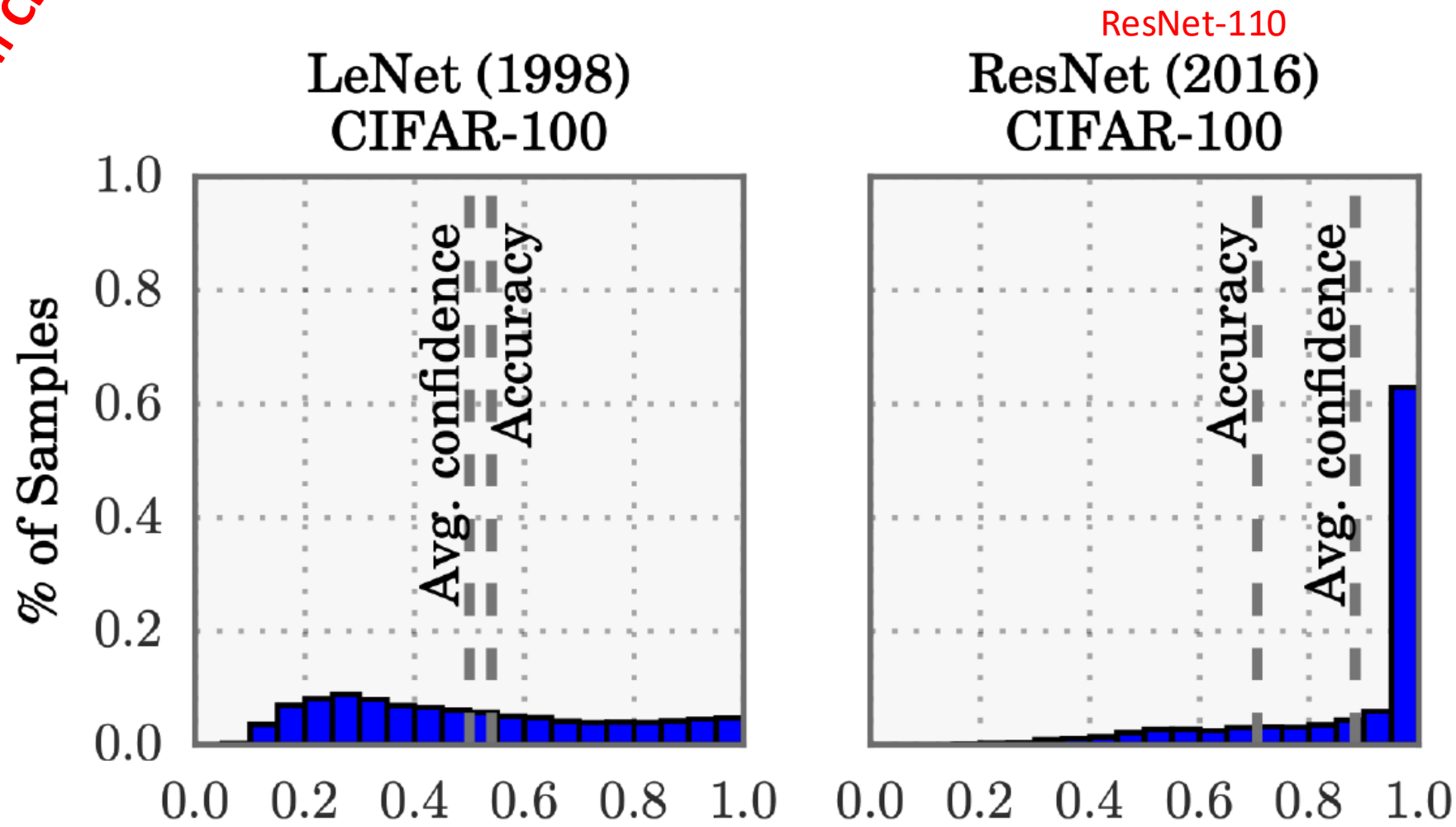
Network f

$f(\phi) \not\subseteq \psi$

- Acknowledgement for slides:
  - CIS 6730 (Computer Aided Verification) Lectures
  - Formal methods in AI: Gagandeep Singh and Madhu Parthasarathy (UIUC)

Previously on CENG7880!

Modern Neural Networks are Overconfident

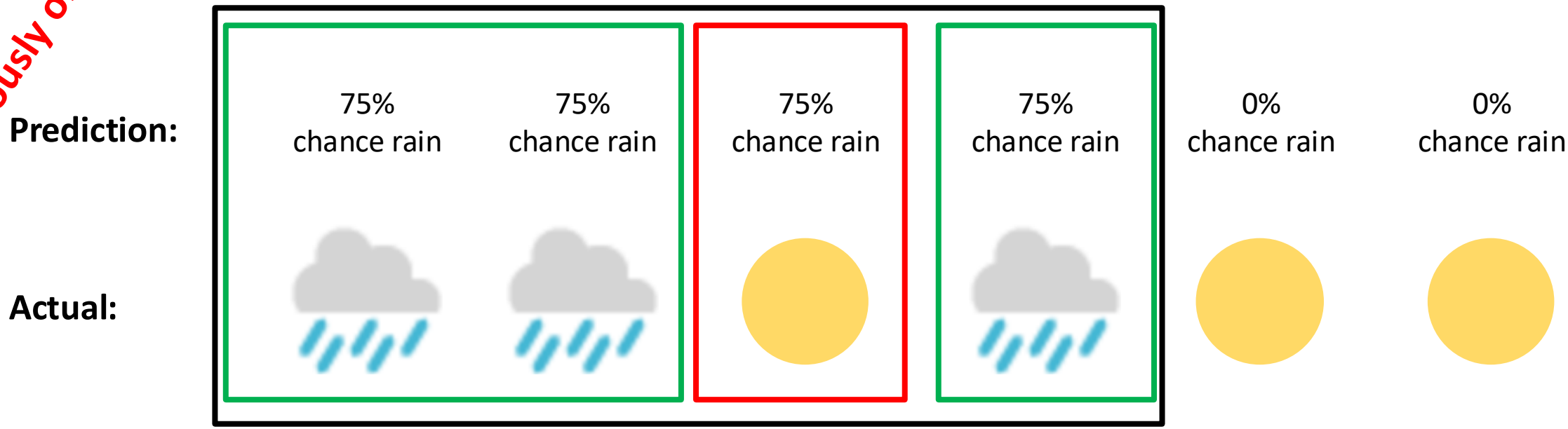


The plots are from this paper: https://arxiv.org/pdf/1706.04599

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

Previously on CENG7880!

Calibrated Prediction

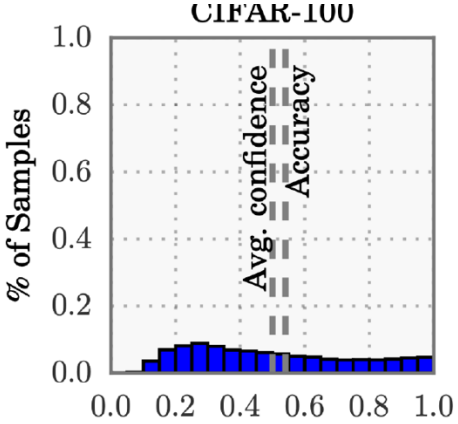


$$\Pr_{p(y^*)}[y^* = \text{rain} \mid \hat{p} = 0.75] = 0.75$$

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

Previously on CENG7880!

Measuring Calibration



- **Recall:** Binned calibration is defined as

$$\Pr_{p(x,y^*)}[\hat{y}(x) = y^* \mid \hat{p}(x) \in P_i] = \text{Conf}(P_i) \qquad (\forall i \in \{1, \ldots, k\})$$

  - Chances of equality holding exactly is effectively zero
  - How to **measure** calibration error (instead of requiring exact calibration)?

- **Idea:** Use mean absolute error (called **expected calibration error**):

$$\text{ECE}(\hat{p}) = \mathbb{E}_{p(P)}\left[\left|\Pr_{p(x,y^*)}[\hat{y}(x) = y^* \mid \hat{p}(x) \in P] - \text{Conf}(P)\right|\right]$$

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

Previously on CENG7880!

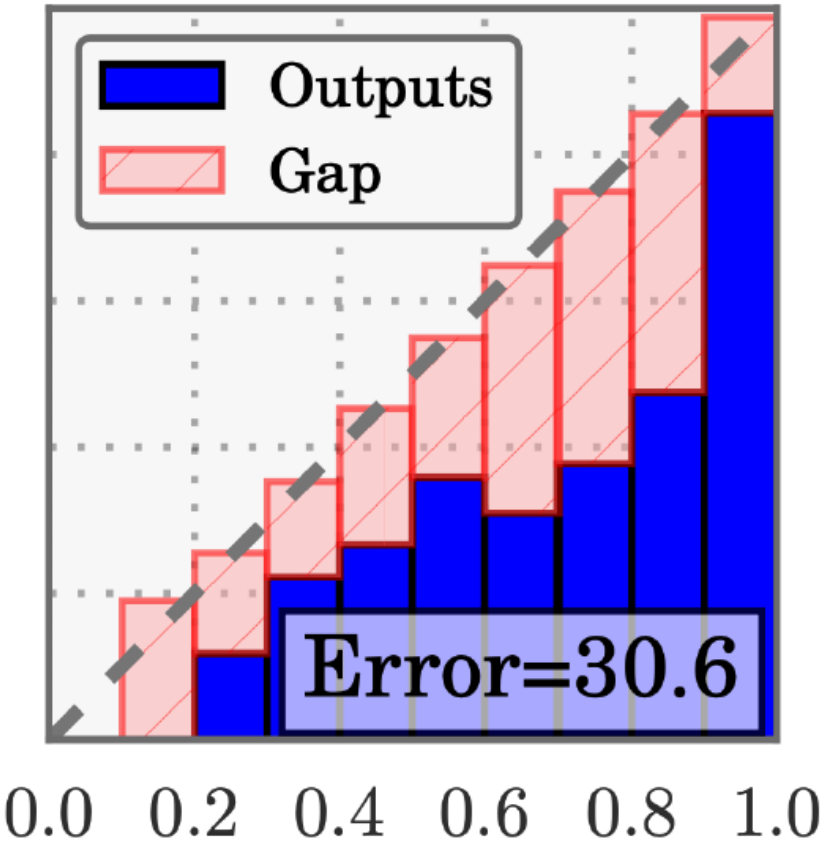
Reliability Diagrams

- For each bin P_i, plot accuracy

$$\Pr_{p(x,y^*)}[\hat{y}(x) = y^* \mid \hat{p}(x) \in P]$$

$$\approx \frac{1}{|B_i|} \sum_{(x,y^*) \in B_i} 1(\hat{y}(x) = y^*)$$

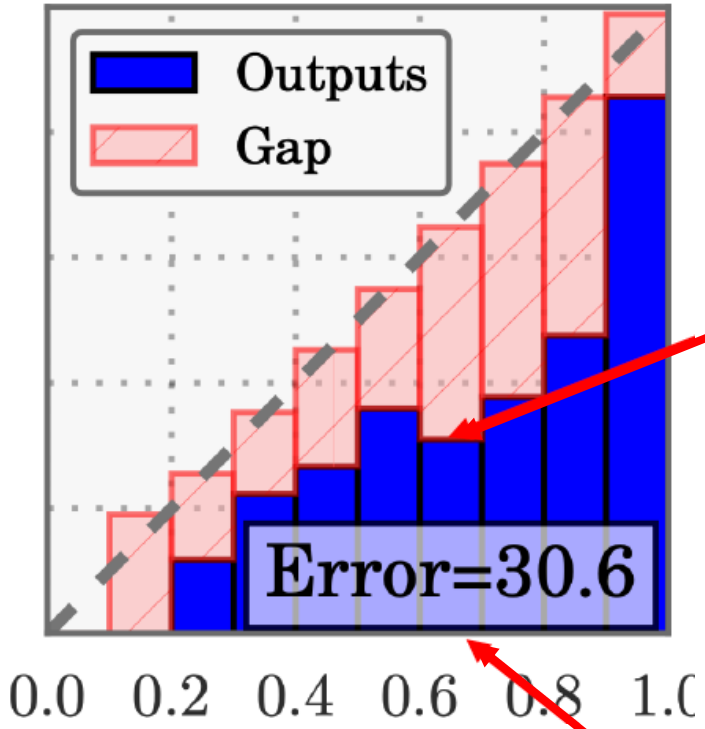
ResNet-110



The plots are from this paper: https://arxiv.org/pdf/1706.04599

Previously on CENG7880!

Histogram Binning



Given x

$$\text{Acc}(P) = \frac{1}{|B|}\sum_{(x',y^*)\in B} 1(\hat{y}(x') = y^*) = 0.35$$

$$\phi(x) = \text{Acc}(P) = 0.35$$

$$\hat{p}(x) = 0.62 \in [0.6,0.7) := P$$

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

Previously on CENG7880!

Temperature Scaling

- Consider the model family

$$\vec{q}_\tau(x) = \text{softmax}\left(\frac{\text{logits}(x)}{\tau}\right)$$

  - Taking $\tau = 1$ recovers the original model: $\vec{q}_1(x) = \vec{p}(x)$
  - Taking $\tau > 1$ decreases confidence ($\tau \to \infty$ yields probabilities equal to $\frac{1}{k}$)
  - Taking $\tau < 1$ increases confidence ($\tau \to 0$ yields probabilities equal in $\{0,1\}$)

- Choose τ to minimize NLL of $\vec{q}_\tau$ on calibration dataset Z
  - Can use grid search to do so (i.e., search over fixed set of choices for τ)

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

Previously on CENG7880!

Conformal Prediction: Goal

- Given a predictor f, modify it to obtain a set predictor $\tilde{f}$ such that it predicts a set.
- In such a way that the correct label is within the set with $1-\alpha$ reliability/confidence:

$$P\left(Y_{test} \in \tilde{f}(X_{test})\right) \geq 1-\alpha$$

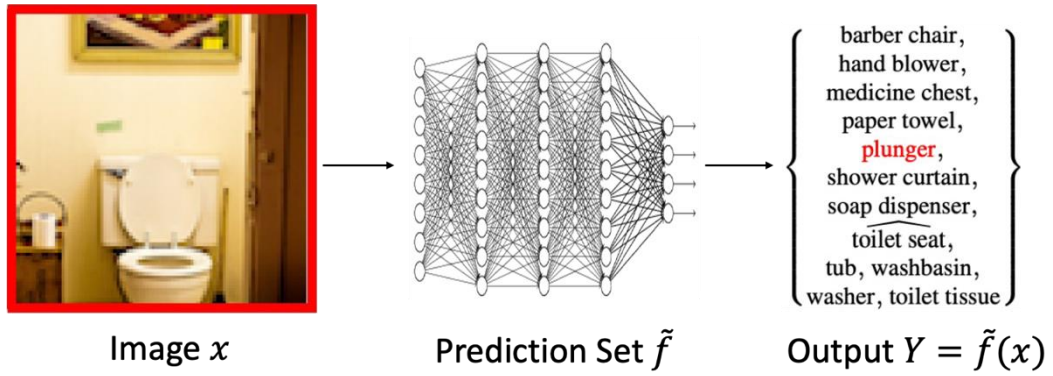


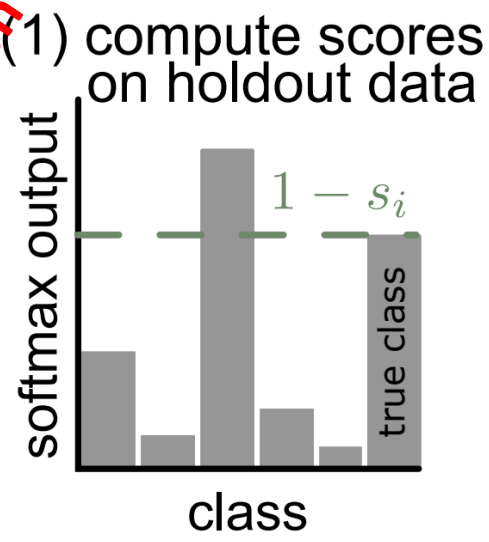
Image x Prediction Set $\tilde{f}$ Output $Y = \tilde{f}(x)$

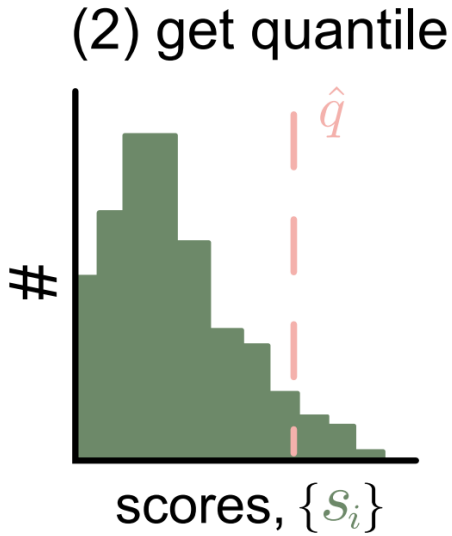
Previously on CENG7880!

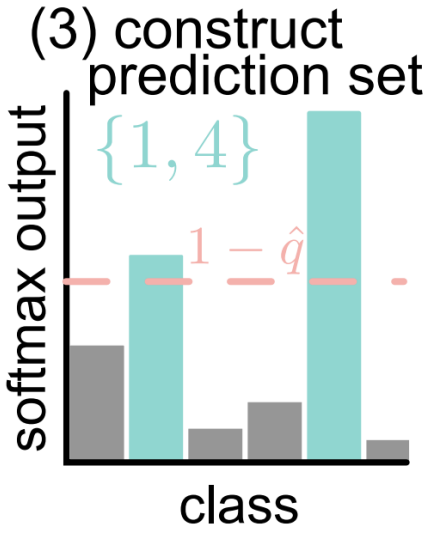
Conformal Prediction Recipe

Recall our goal:

$$P\left(Y_{test} \in \tilde{f}(X_{test})\right) \geq 1 - \alpha$$







(1) Score the held-out data

$$s_i = 1 - f(y_i^*|x_i), \qquad i = 1, \ldots, n$$

low softmax on the true class ⇒ large score

(2) Threshold: a corrected quantile

$$\hat{q} = s_{(k)}, \qquad k = \lceil (n+1)(1-\alpha) \rceil$$

k-th smallest score ≈ the $(1-\alpha)$ quantile

(3) Predict a set for a new input

$$\tilde{f}(x_{test}) = \{y : f(y|x_{test}) \geq 1 - \hat{q}\}$$

a level set of f with threshold $\tau = 1 - \hat{q}$

Interpretation: with $\alpha = 0.1$, at least 90% of held-out true-class softmax values lie above $1 - \hat{q}$ ⇒ keeping every class above $1 - \hat{q}$ captures the true class ≥ 90% of the time.

Material: https://arxiv.org/pdf/2107.07511

Previously on CENG7880!

Conformal Prediction for Any/All Problems

- Whether the inputs/outputs are discrete/continuous or not



(Softmax probabilities in the previous example)

Fig & material: https://arxiv.org/pdf/2107.07511

Adaptive Prediction Sets: The Algorithm

Previously on CENG 7880!

Sort the classes of an input x by softmax:

$$f(y_1|x) \geq f(y_2|x) \geq \cdots \geq f(y_K|x)$$

(1) Score: Sum up to and including the true class

$$s_i = \sum_{j=1}^{r} f(y_j|x_i), \quad \text{where } y_r = y_i^*$$

(2) Threshold: exactly as before

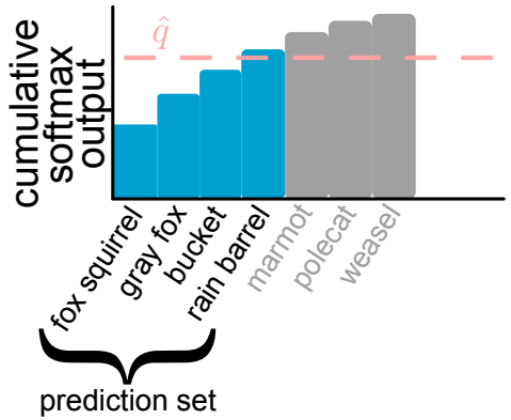
$$\hat{q} = s_{(k)}, \qquad k = \lceil (n+1)(1-\alpha) \rceil$$

(3) Predict: add classes in sorted order while the mass stays $\leq \hat{q}$

$$\tilde{f}(x_{test}) = \{y_1, \ldots, y_r : \sum_{j=1}^{r} f(y_j|x_{test}) \leq \hat{q}\}$$

Same method with new score: $\tilde{f}(x) = \{y : s(x,y) \leq \hat{q}\}$

Material: https://arxiv.org/pdf/2107.07511 (Sec. 2.1);
APS: Romano, Sesia & Candès, NeurIPS 2020



Example (illustrative, $\hat{q} = 0.85$)

Class	$f(y_j\|x)$	Cumul.	In set?
fox squirrel	0.40	0.40	yes
gray fox	0.15	0.55	yes
bucket	0.13	0.68	yes
rain barrel	0.12	0.80	yes
marmot	0.08	0.88	no
polecat	0.06	0.94	no
weasel	0.06	1.00	no

- First 4 classes form the set (as in the figure)
- Confident input, e.g. $f(y_1|x) = 0.9 \Rightarrow$ set of size 1
- Original APS randomizes the boundary class (omitted)

Previously on CENG7880!

Conformal Prediction for Regression

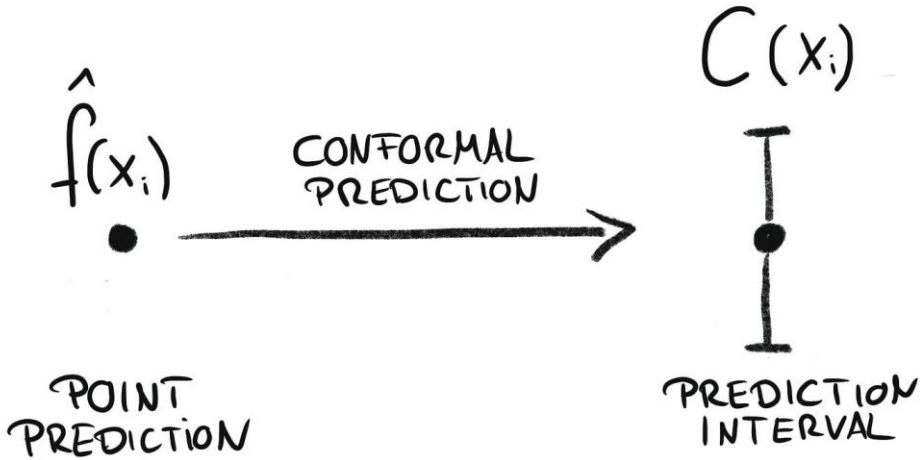
Goal: Turn a point prediction $\hat{f}(x)$ into an interval $C(x)$ that contains the true y with probability $\geq 1 - \alpha$.

In other words: the true value lies in

$$C(x) = \left[\hat{f}(x) - \hat{q}, \hat{f}(x) + \hat{q}\right]$$

with probability $\geq 1 - \alpha$.

- $\hat{q}$: half-width, calibrated on held-out data (next slide)
- α: user-chosen error rate, e.g., $\alpha = 0.1 \Rightarrow$ 90% intervals
- Applicable to any regressor; no retraining.



https://mindfulmodeler.substack.com/p/week-3-conformal-prediction-for-regression

Verification+Calibration+Conformal Prediction Tutorial

https://colab.research.google.com/drive/1GT7livzqvPAaw5-goI8tgKddcR0NRtke?usp=sharing

By Ugur Yalcin and Melih Darcan

Agenda

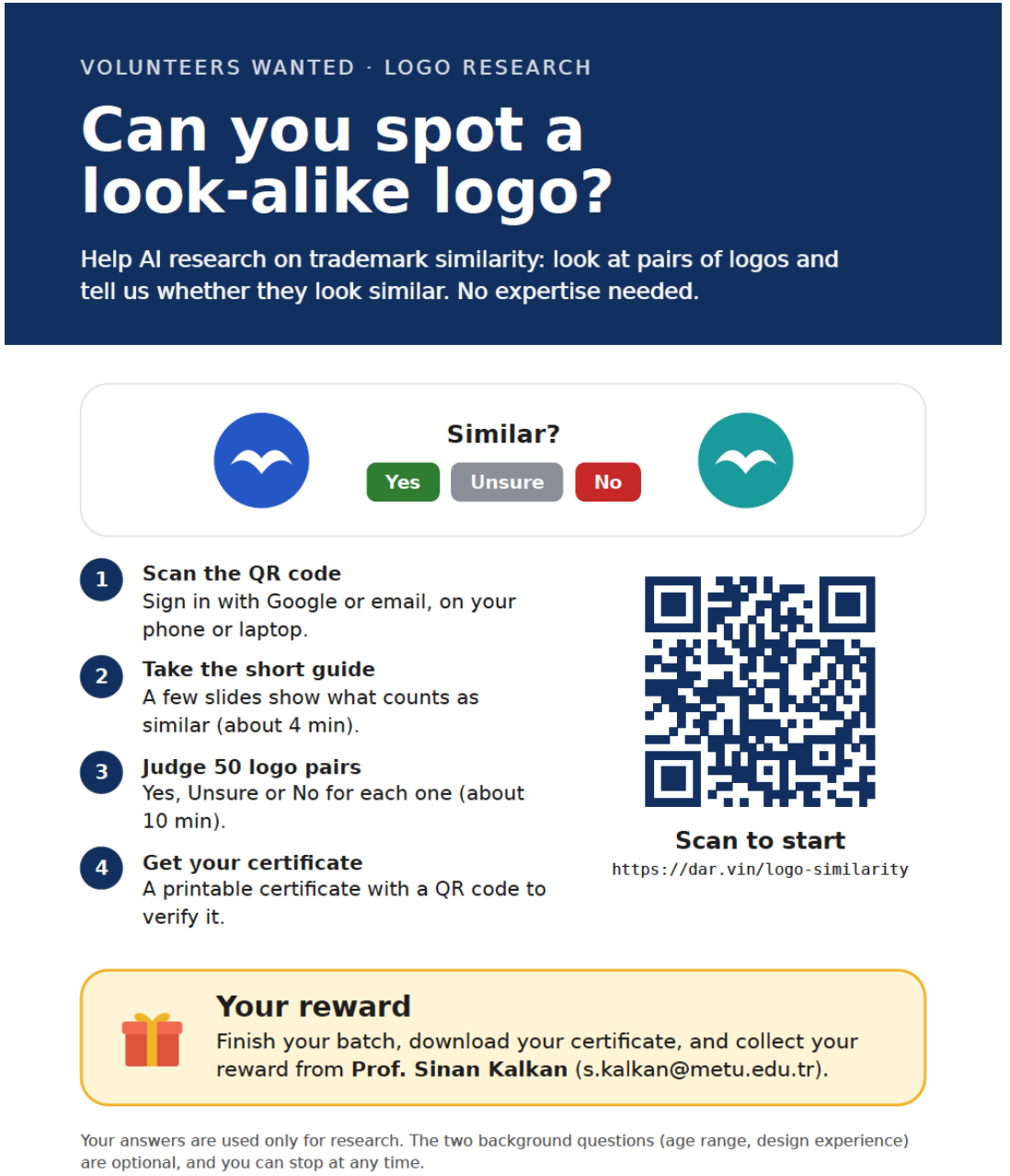
- Robustness
  - Uncertainty
    - Uncertainty Types & Quantification
  - Robustness in LLMs/VLMs

Administrative Notes

- Selecting papers
  - Feedback provided
  - Next steps:
    - **1. Milestone (November 22, midnight):**
      - Read & understand the paper
      - Download the datasets
      - Prepare the Readme file excluding the results & conclusion
    - **2. Milestone (December 6, midnight)**
      - The results of the first experiment
    - **3. Milestone (January 4, midnight)**
      - Final report (Readme file)
      - Repo with all code & trained models

Help needed

- https://203-101-229-244.sslip.io/login



Predictive Uncertainty

Predictive Uncertainty

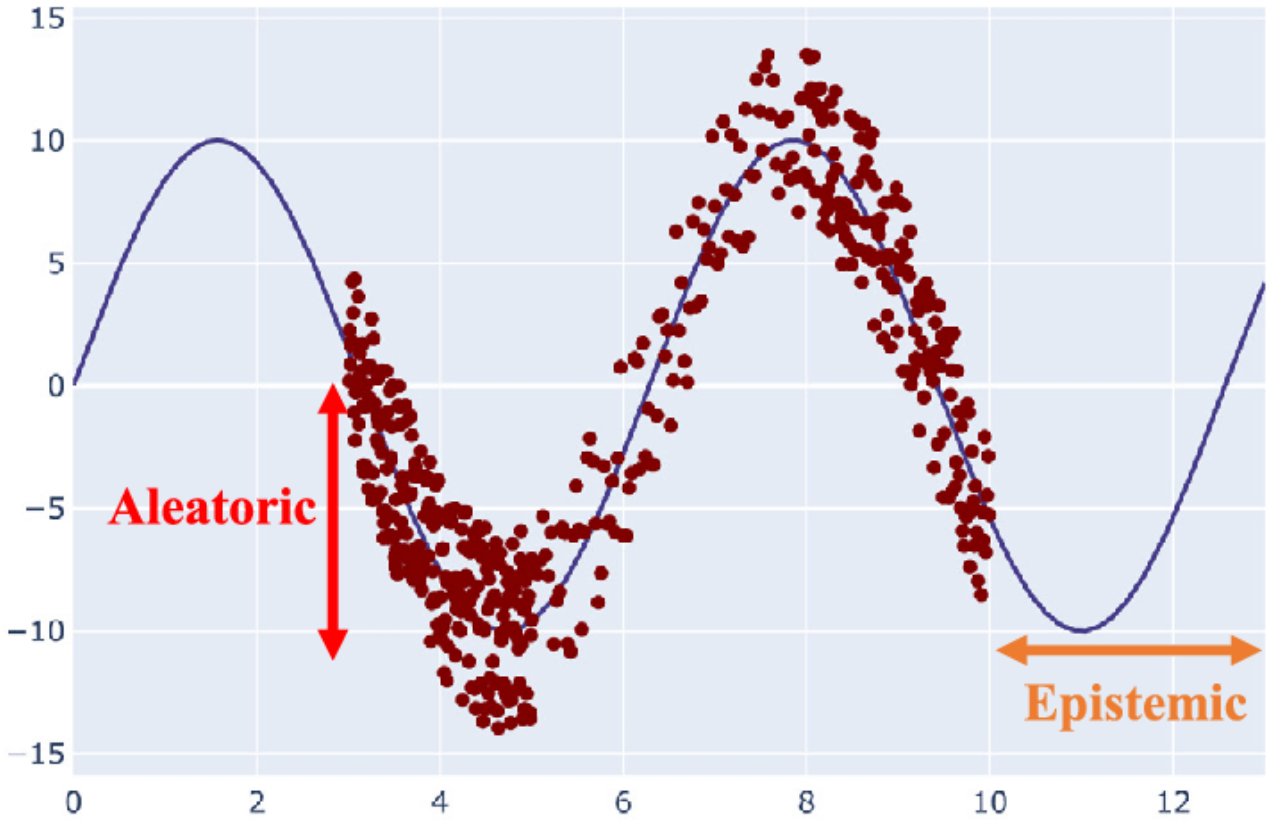
- **Goal:** What is the distribution of $y - f_{\hat{\beta}}(x)$?

- Useful for decision-making
  - Uncertain ➔ patient should be seen by a doctor
  - Uncertain ➔ robot should avoid potential obstacle

- However, aggregates multiple sources of uncertainty

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

Types of Uncertainty



Epistemic ('knowledge') Uncertainty

- **Lack of data**: When dataset is small or not representative.
- **Lack of model information**: Without knowing the true data generating process, we make assumptions about the suitable model family, architecture etc.

Aleatoric ('randomness') Uncertainty

- Inherent noise in the data or the labels

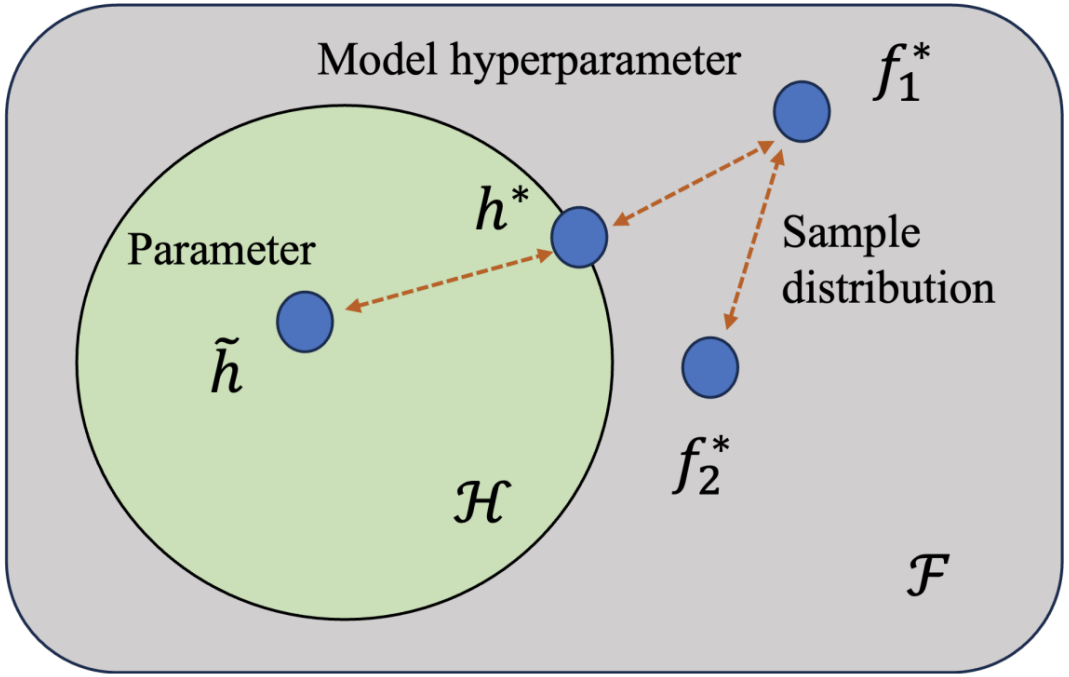
Fig: Abdar et al., A review of uncertainty quantification in deep learning: Techniques, applications and challenges, 2021

Sources of model (epistemic) uncertainty

- $\mathcal{F}$: Full hypothesis space. E.g., the functions that can be represented by Transformers.
- $\mathcal{H}$: A subspace of $\mathcal{F}$; e.g., functions that can be represented with specific model hyperparameters.
- h^*: Theoretical optimal solution in $\mathcal{H}$.

$$h^* = \arg\min_{h \in \mathcal{H}} R(h), \quad \text{where}$$

$$R(h) = \int l(y, h(\boldsymbol{x}))p(\boldsymbol{x}, y)d\boldsymbol{x}dy.$$



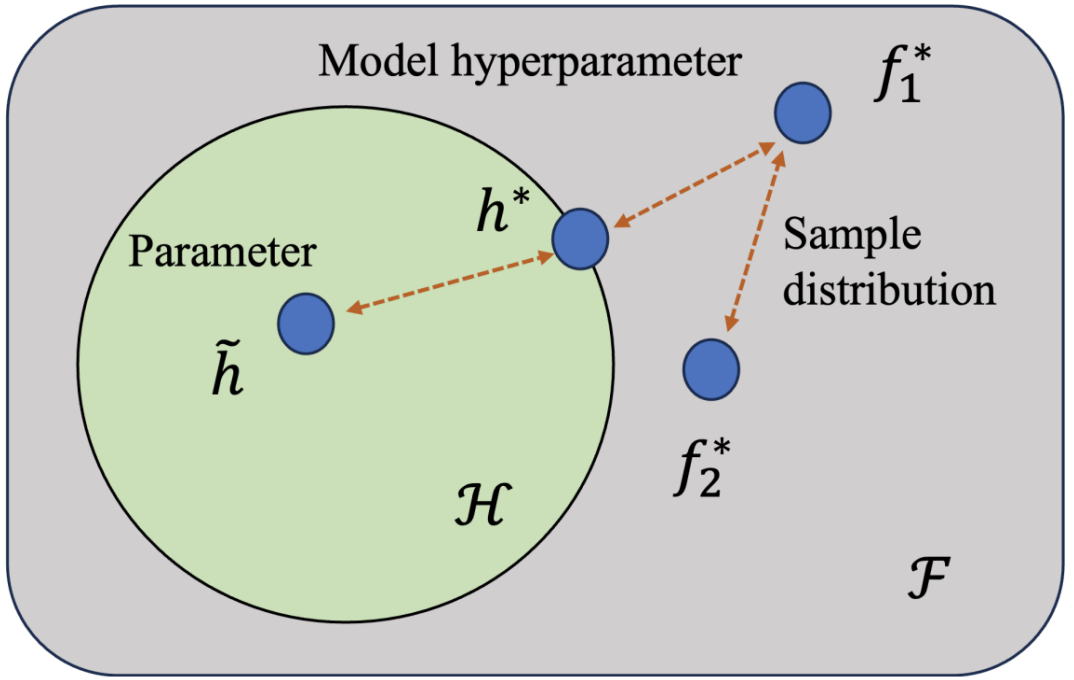
Fig, Eqs and content: A survey on uncertainty quantification methods for deep learning, 2023. arxiv.org/pdf/2302.13425

Sources of model (epistemic) uncertainty

https://arxiv.org/pdf/2302.13425

- $f_1^* \in \mathcal{F}$: Optimal solution with respect to one sample distribution $p_1(\boldsymbol{x}, y)$.
- $f_2^* \in \mathcal{F}$: Optimal solution with respect to one sample distribution $p_2(\boldsymbol{x}, y)$.

- $\tilde{h}$: Solution in $\mathcal{H}$ that is learned on a training set D_{tr} drawn from $p_1(\boldsymbol{x}, y)$.



$$\tilde{h} = \arg\min_{h \in \mathcal{H}} R_{emp}(h), \text{ where } R_{emp}(h) = \frac{1}{|\mathcal{D}_{tr}|} \sum_{(\boldsymbol{x}_i, y_i) \in \mathcal{D}_{tr}} l(y_i, h(\boldsymbol{x}_i)).$$

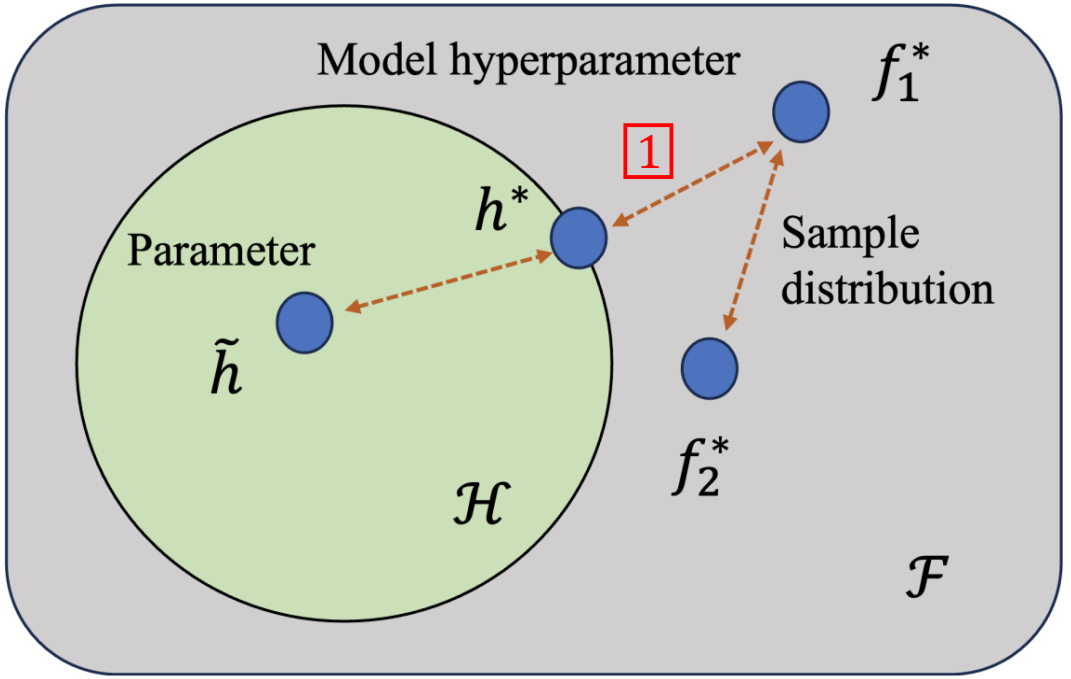
Fig, Eqs and content: A survey on uncertainty quantification methods for deep learning, 2023. arxiv.org/pdf/2302.13425

Sources of model (epistemic) uncertainty

Table 1. Comparison of different types of model uncertainty in the supervised learning setting

	Model uncertainty sources	Corresponding notation in supervised learning
1	Choice of model hyper-parameter	Optimal solution h^* within $\mathcal{H}$ does not align with theoretical optimal f^* in $\mathcal{F}$

https://arxiv.org/pdf/2302.13425



This discrepancy will occur even if you have the perfect training algorithm.

Fig, table and content: A survey on uncertainty quantification methods for deep learning, 2023. arxiv.org/pdf/2302.13425

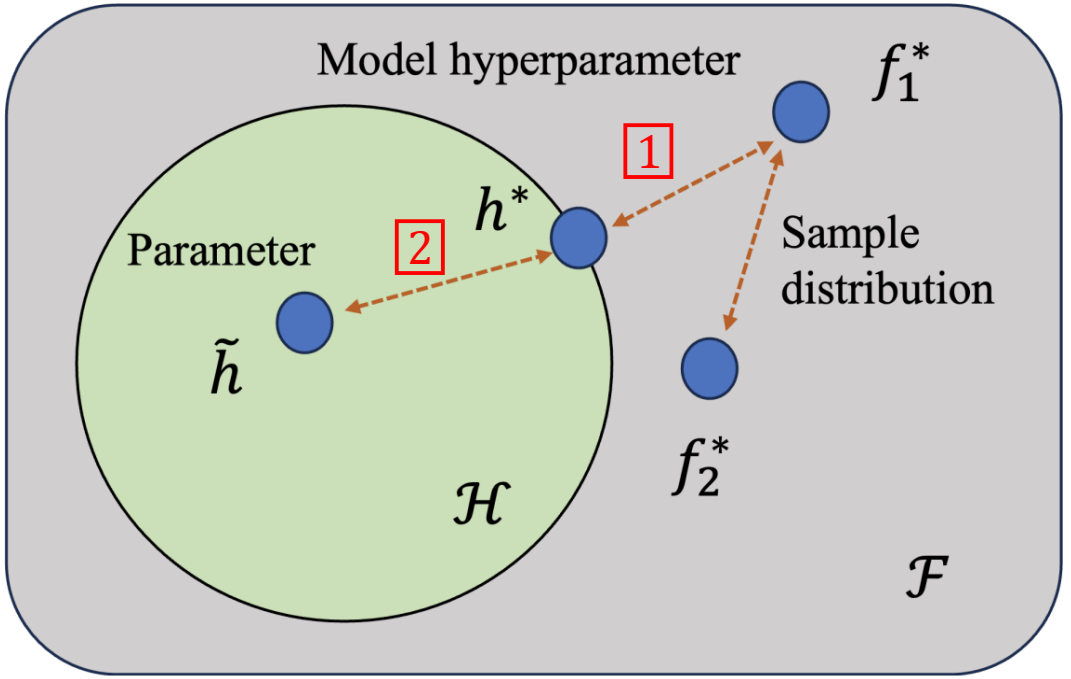
Sources of model (epistemic) uncertainty

Table 1. Comparison of different types of model uncertainty in the supervised learning setting

https://arxiv.org/pdf/2302.13425

Model uncertainty sources	Corresponding notation in supervised learning
Choice of model hyper-parameter	Optimal solution h^* within $\mathcal{H}$ does not align with theoretical optimal f^* in $\mathcal{F}$
Model parameter learning	Learned solution $\tilde{h}$ does not align with optimal h^* in $\mathcal{H}$

2



Due to insufficient data or imperfect learning algorithm.

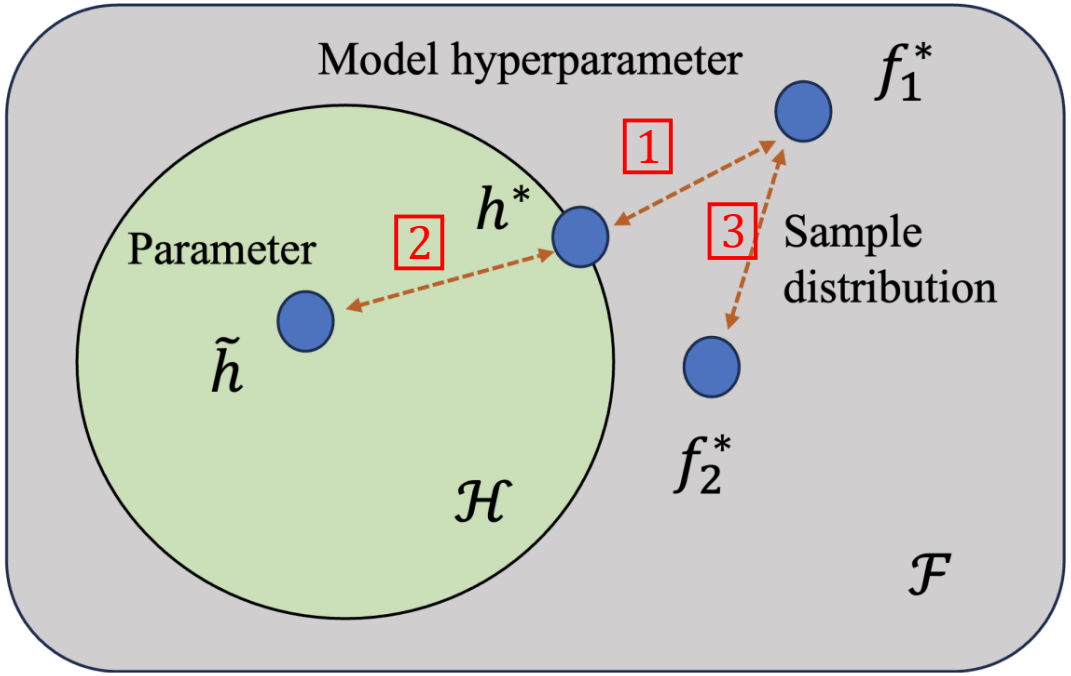
Fig, table and content: A survey on uncertainty quantification methods for deep learning, 2023. arxiv.org/pdf/2302.13425

Sources of model (epistemic) uncertainty

https://arxiv.org/pdf/2302.13425

Table 1. Comparison of different types of model uncertainty in the supervised learning setting

	Model uncertainty sources	Corresponding notation in supervised learning
1	Choice of model hyper-parameter	Optimal solution h^* within $\mathcal{H}$ does not align with theoretical optimal f^* in $\mathcal{F}$
2	Model parameter learning	Learned solution $\tilde{h}$ does not align with optimal h^* in $\mathcal{H}$
3	Different sample distributions in learning and inference	Theoretical optimum f_1^* and f_2^* mismatch under different sample distribution $p(\boldsymbol{x}, y)$



- Due to different sample distributions between training and inference.
- A model trained on $p_1(\boldsymbol{x}, y)$ will make errors on samples from $p_2(\boldsymbol{x}, y)$.
  - Out of distribution samples.
  - Samples far from training set.

Fig, table and content: A survey on uncertainty quantification methods for deep learning, 2023. arxiv.org/pdf/2302.13425

Sources of data (aleatoric) uncertainty

- Owing to inherent data randomness, noise or class/label confusion
- Irreducible even with more training data

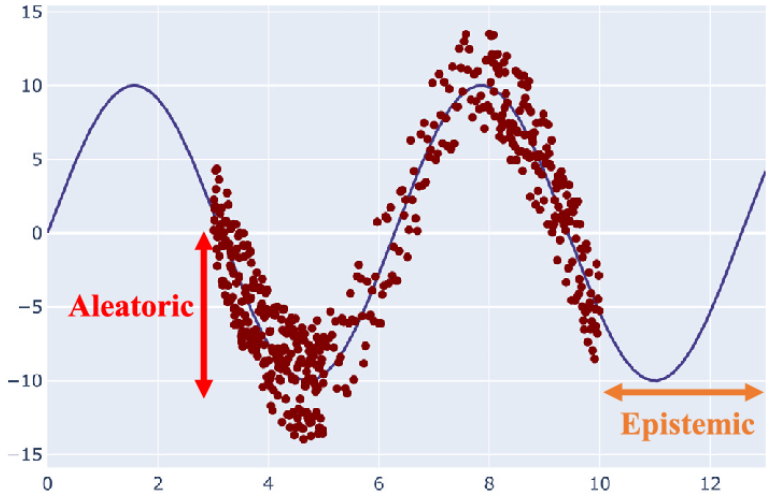
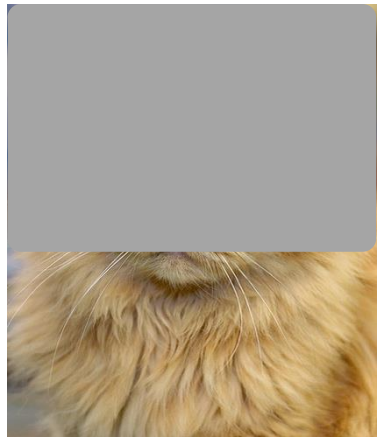


Fig: Abdar et al., A review of uncertainty quantification in deep learning: Techniques, applications and challenges, 2021



Sample from Imagenet

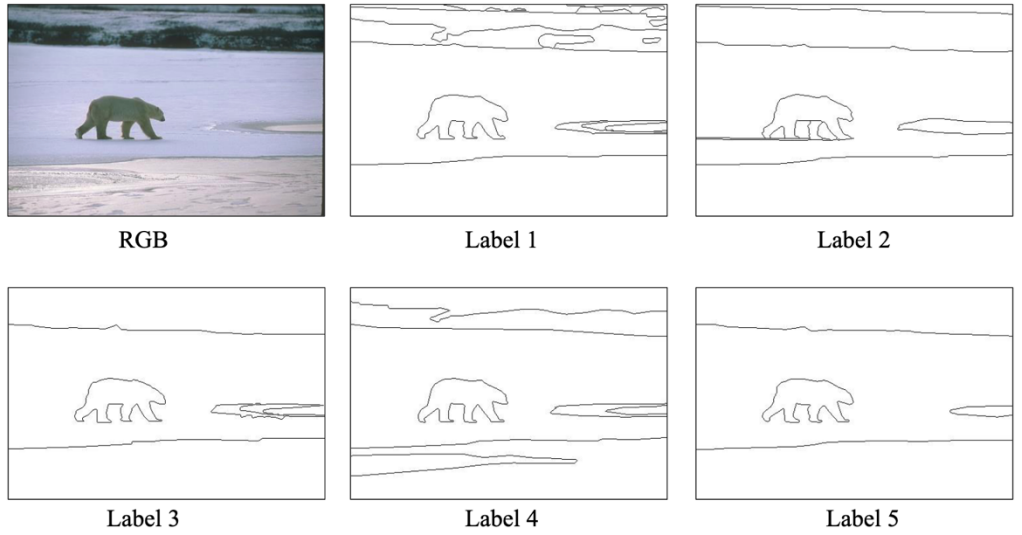


Figure 1.3: An example of the multi-label challenge in edge detection: a single input image and its five different ground truth edge maps annotated by five different human annotators. The image and labels are taken from the BSDS dataset [1].

From Bedrettin Cetinkaya's PhD Thesis

Sources of data (aleatoric) uncertainty

- Owing to inherent data randomness, noise or class/label confusion
- Irreducible even with more training data

Two types:

- Homoscedastic (homogeneous variances) noise: Constant observation noise for all samples.
- Heteroscedastic (heterogeneous variances) noise: Input dependent noise.

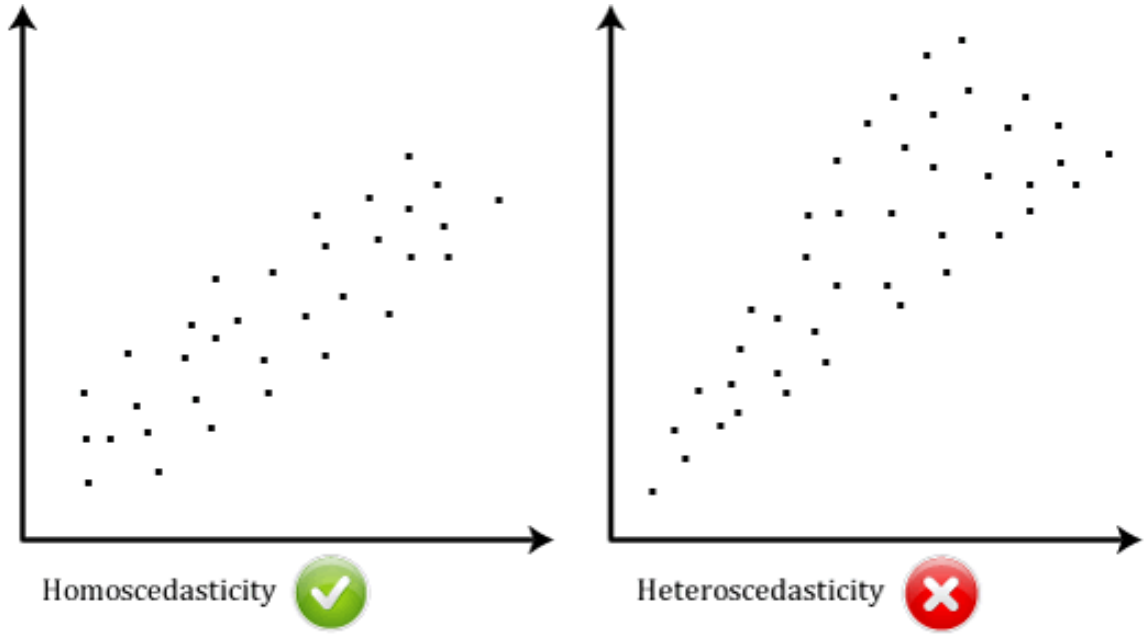


Fig: https://stats.stackexchange.com/questions/76151/what-is-an-intuitive-explanation-of-why-we-want-homoskedasticity-in-a-regression

Motivation: Active Learning

- **Goal:** Will obtaining additional information help make better decisions?

- **Example**
  - Robot is not sure if an object is a fork or a spoon
  - Is it worth moving closer to get a better look?



yes!
epistemic uncertainty



no!
aleatoric uncertainty

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

Aleatoric vs. Epistemic Uncertainty

- In general, the **residual error** decomposes as

$$y - f_{\hat{\beta}(Z)}(x)$$

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

Aleatoric vs. Epistemic Uncertainty

- In general, the **residual error** decomposes as

$$\underbrace{y - f_{\hat{\beta}(Z)}(x)}_{\text{Predictive uncertainty}} = \underbrace{\left(y - f_{\beta^*}(x)\right)}_{\text{Aleatoric uncertainty}} + \underbrace{\left(f_{\beta^*}(x) - f_{\hat{\beta}(Z)}(x)\right)}_{\text{Epistemic uncertainty}}$$

  - **Aleatoric uncertainty:** Error of best possible model f_{β^*}
  - **Epistemic uncertainty:** Error of our model $f_{\hat{\beta}(Z)}$ vs. f_{β^*}

- How can we disentangle the two?

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

Uncertainty Quantification in Deep/Machine Learning

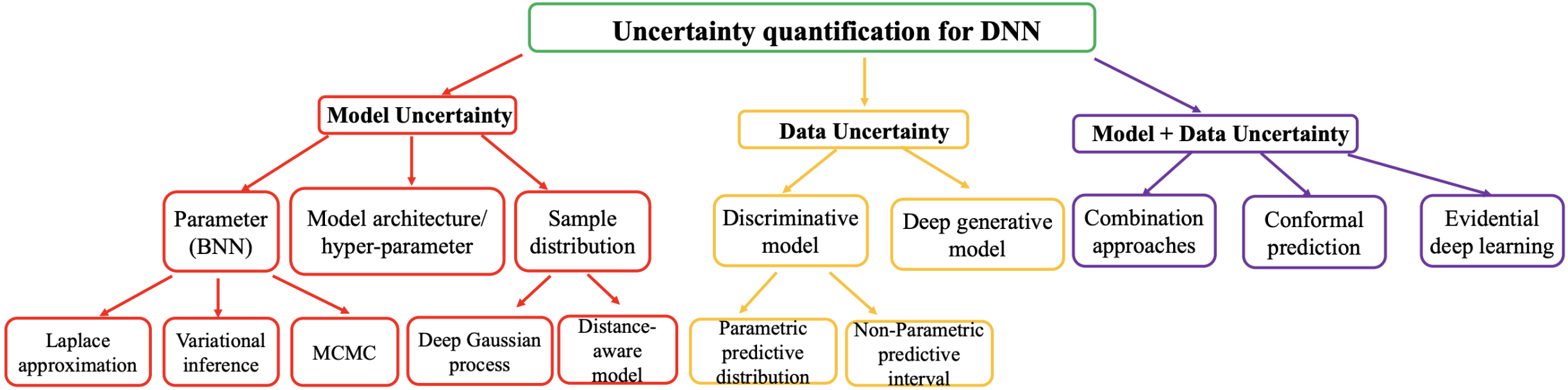


Fig. 5. A taxonomy for existing literature on UQ for DNN.

Fig, table and content: A survey on uncertainty quantification methods for deep learning, 2023. arxiv.org/pdf/2302.13425

Aleatoric vs. Epistemic Uncertainty

- In general, we have

$$y - f_{\hat{\beta}}(x) = \left(y - f_{\beta^*}(x)\right) + \left(f_{\beta^*}(x) - f_{\hat{\beta}(Z)}(x)\right)$$

- **Hard to disentangle**
  - We directly observe the predictive uncertainty $y - f_{\hat{\beta}}(x)$
  - But we don't know β^*

- **General strategy (statistics):** Pretend $\mathbb{E}_Z\left[\hat{\beta}(Z)\right] = \beta^*$, and disentangle
  - Works in practice even if it feels circular

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

Estimating Epistemic Uncertainty

Epistemic Uncertainty

- The **epistemic uncertainty** is

$$\text{Epistemic}(x) = f_{\beta^*}(x) - f_{\widehat{\beta}(Z)}(x)$$

  - Here, $\text{Epistemic}(x)$ is a random function of the random variable $Z \sim p^n$
  - Thus, $\text{Epistemic}(x)$ is itself a random variable

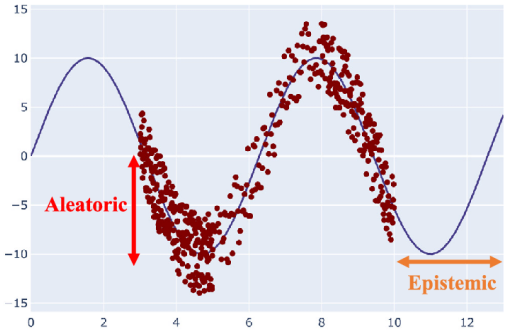
- **Goal:** Estimate the distribution of $\text{Epistemic}(x)$

But we don't know β^*

- **Assumption:** Our model is **unbiased**: $\mathbb{E}_Z\left[f_{\widehat{\beta}(Z)}(x)\right] = f_{\beta^*}(x)$

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

What if we know $p(x)$?



- $\{f_{\beta^*}(x) - \hat{f}_i(x)\}_{i=1}^{k}$ are i.i.d. samples from $\text{Epistemic}(x)$

- By our unbiasedness assumption:

Same architecture trained on different subsets/versions of Z.

$$f_{\beta^*}(x) = \mathbb{E}_Z\left[f_{\hat{\beta}(Z)}(x)\right] \approx k^{-1}\sum_{i=1}^{k}\hat{f}_i(x) := \hat{\mu}(x)$$

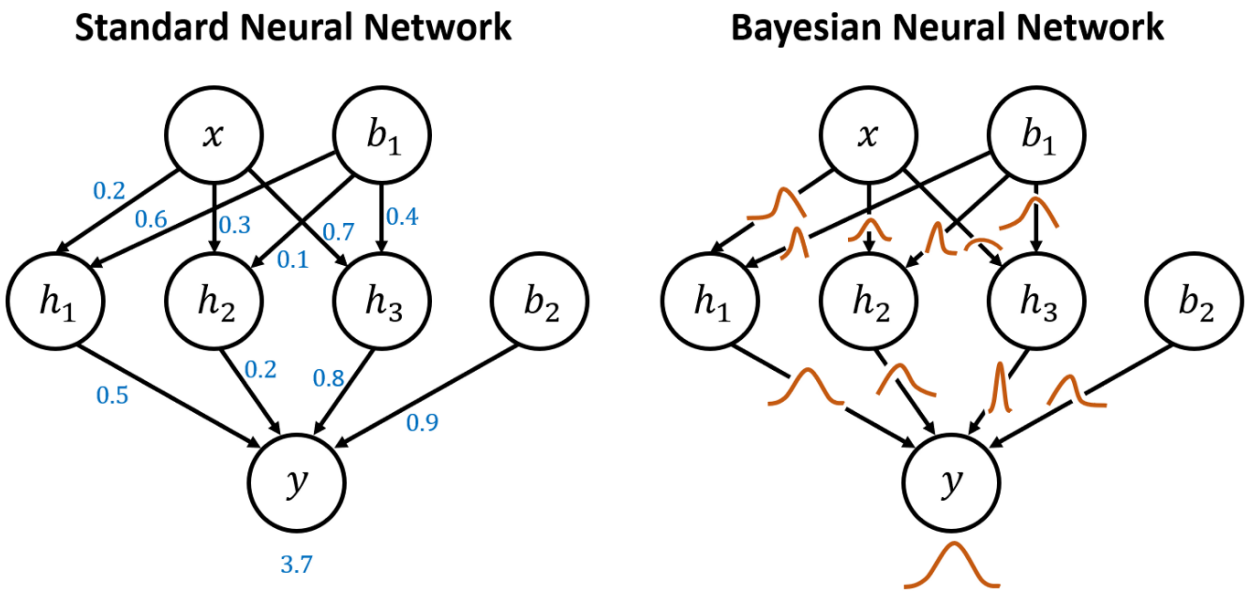
- $\{\hat{\mu}(x) - \hat{f}_i(x)\}_{i=1}^{k}$ are approximately i.i.d. samples from $\text{Epistemic}(x)$
  - **Problem:** We cannot take unlimited samples from p
  - Only have a single training dataset Z!

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

From Multiple Estimations to Epistemic Uncertainty

- $\{\hat{\mu}(x) - \hat{f}_i(x)\}_{i=1}^{k}$ are approximately i.i.d. samples from $\mathrm{Epistemic}(x)$
- Given $\{\hat{\mu}(x) - \hat{f}_i(x)\}_{i=1}^{k}$, we can quantify its dispersion to estimate epistemic uncertainty
- We can look at:
  - Variance of $\{\hat{f}_i(x)\}_{i=1}^{k}$
  - Entropy of $\{\hat{f}_i(x)\}_{i=1}^{k}$

Methods for Estimating Epistemic Uncertainty



- With a Bayesian Neural Network, we can sample different "models" from a distribution over the weights.

Fig: https://medium.com/@costaleirbag/a-first-insight-into-bayesian-neural-networks-bnn-c767551e9526

Methods for Estimating Epistemic Uncertainty

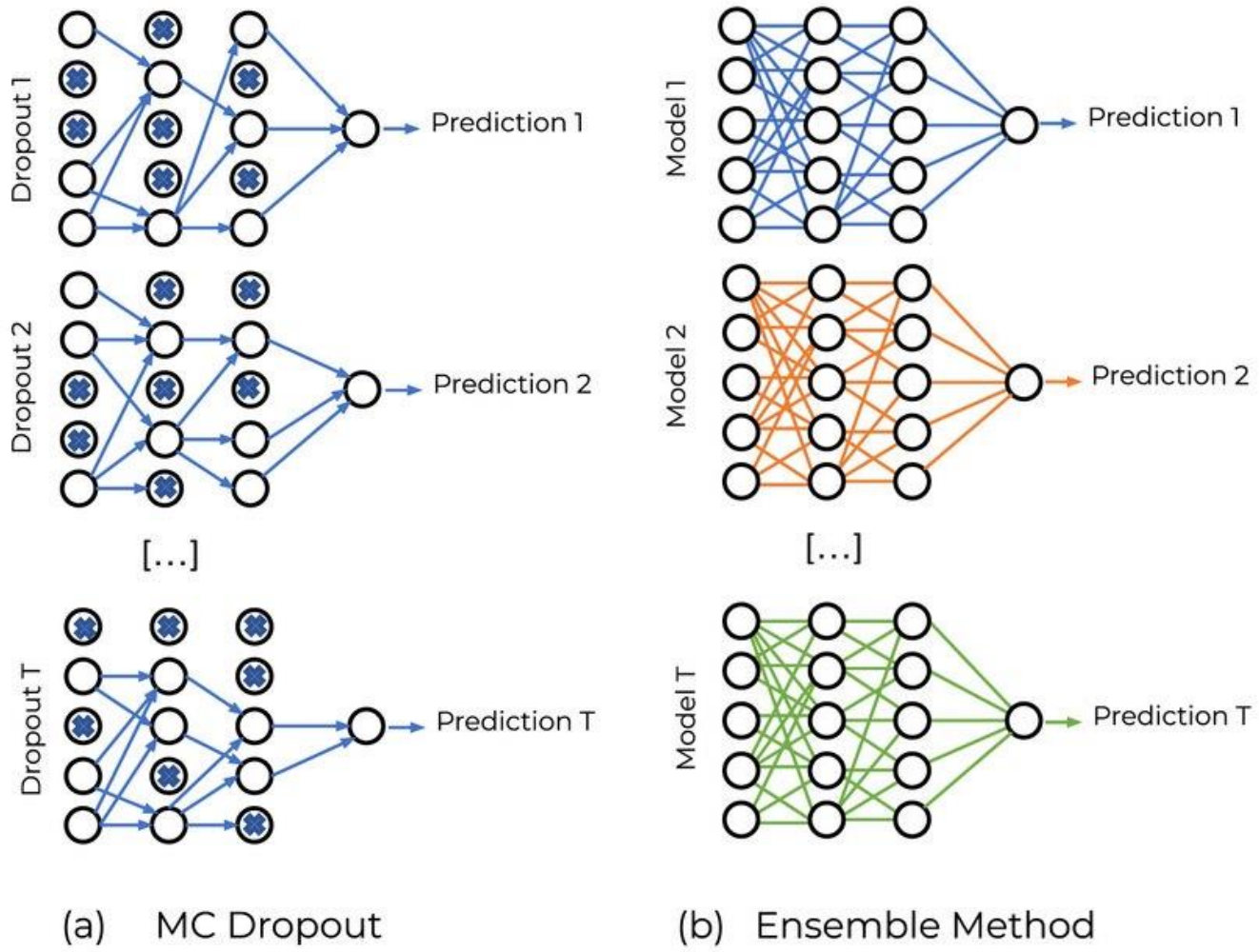


Fig: https://www.researchgate.net/figure/a-MC-Dropout-at-inference-time-The-T-predictions-are-obtained-from-Dropout-of_fig10_359970124

Methods for Estimating Epistemic Uncertainty: Monte Carlo Dropout

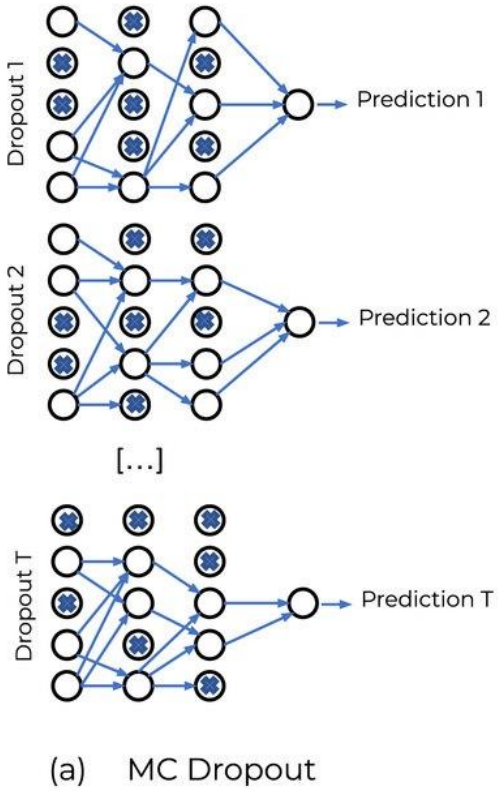
Recall: Epistemic Uncertainty is dispersion of $\{\hat{f}_i(x)\}_{i=1}^{k}$

For classification problems:

For classification this can be approximated using Monte Carlo integration as follows:

$$p(y = c|\mathbf{x}, \mathbf{X}, \mathbf{Y}) \approx \frac{1}{T} \sum_{t=1}^{T} \text{Softmax}(\mathbf{f}^{\widehat{\mathbf{W}}_t}(\mathbf{x})) \quad (3)$$

with T sampled masked model weights $\widehat{\mathbf{W}}_t \sim q_\theta^*(\mathbf{W})$, where $q_\theta(\mathbf{W})$ is the Dropout distribution [6]. The uncertainty of this probability vector $\mathbf{p}$ can then be summarised using the entropy of the probability vector: $H(\mathbf{p}) = -\sum_{c=1}^{C} p_c \log p_c$. For regression this epistemic uncertainty is captured



"What uncertainties do we need for computer vision?"
https://arxiv.org/pdf/1703.04977

Methods for Estimating Epistemic Uncertainty: Monte Carlo Dropout

Clarification with the text:

- σ^2 is aleatoric uncertainty
- The second term is the epistemic uncertainty.

$$\mathrm{Var}(\mathbf{f}^W(\mathbf{x})) = E[\mathbf{f}^W(\mathbf{x})^2] - (E[\mathbf{f}^W(\mathbf{x})])^2$$

For regression problems:

probability vector: $H(\mathbf{p}) = -\sum_{c=1}^{C} p_c \log p_c$. For regression this epistemic uncertainty is captured by the predictive variance, which can be approximated as:

$$\mathrm{Var}(\mathbf{y}) \approx \sigma^2 + \frac{1}{T}\sum_{t=1}^{T} \mathbf{f}^{\widehat{\mathbf{W}}_t}(\mathbf{x})^T \mathbf{f}^{\widehat{\mathbf{W}}_t}(\mathbf{x}_t) - E(\mathbf{y})^T E(\mathbf{y}) \quad (4)$$

with predictions in this epistemic model done by approximating the predictive mean: $E(\mathbf{y}) \approx \frac{1}{T}\sum_{t=1}^{T} \mathbf{f}^{\widehat{\mathbf{W}}_t}(\mathbf{x})$. The first term in the predictive variance, σ^2, corresponds to the amount of noise inherent in the data (which will be explained in more detail soon). The second part of the predictive variance measures how much the model is uncertain about its predictions – this term will vanish when we have zero parameter uncertainty (i.e. when all draws $\widehat{\mathbf{W}}_t$ take the same constant value).

"What uncertainties do we need for computer vision?"
https://arxiv.org/pdf/1703.04977

Recall: Epistemic Uncertainty is dispersion of $\{\hat{f}_i(x)\}_{i=1}^{k}$

Estimating Aleatoric Uncertainty

Methods for Estimating Aleatoric Uncertainty

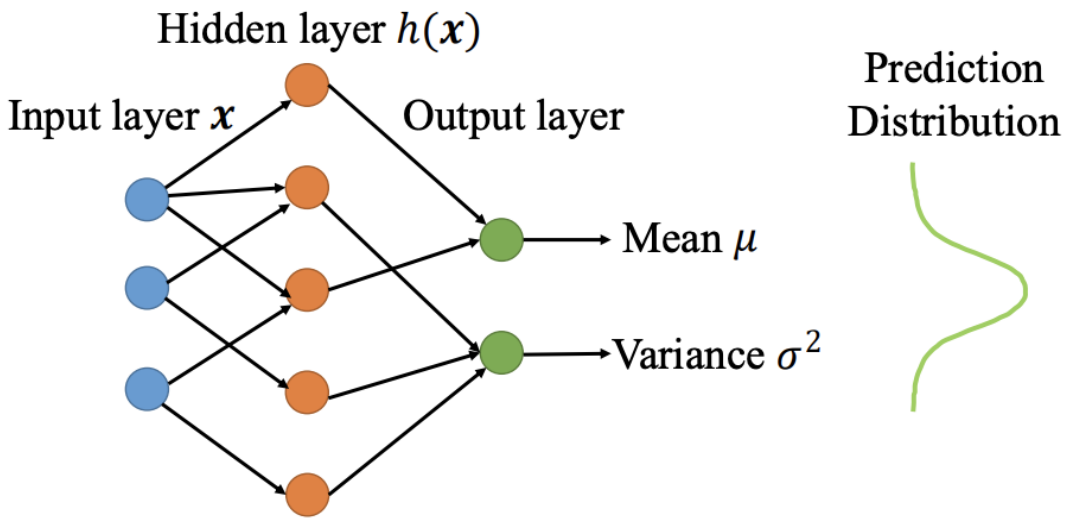
For regression problems:

- Aleatoric uncertainty is modeled by the distribution:

$$p(y|\boldsymbol{x}, \boldsymbol{\theta})$$

- In a regression problem, we can obtain this as:

$$p(y|\boldsymbol{x}, \boldsymbol{\theta}) = \mathcal{N}(f_{\boldsymbol{\theta}}(\boldsymbol{x}), \sigma_{\boldsymbol{\theta}}(\boldsymbol{x}))$$

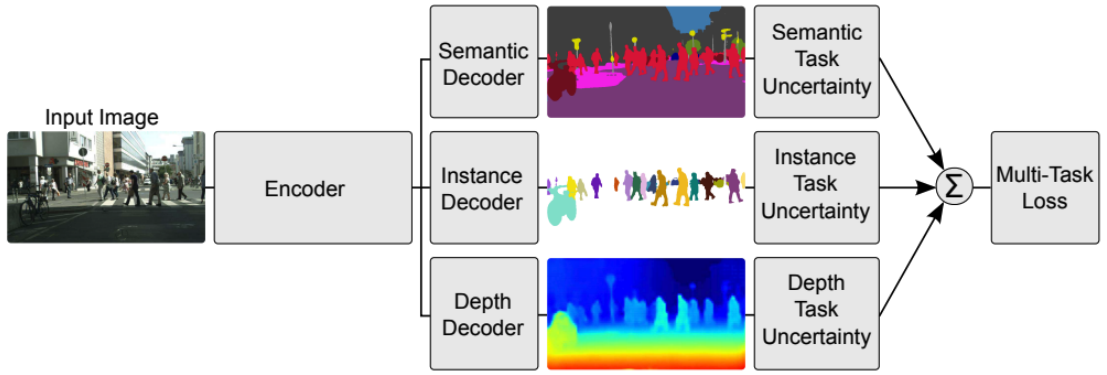


(a) Prediction distribution example.

$$\mathcal{L}_{\mathrm{NN}}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{2\sigma_{\boldsymbol{\theta}}^2(\boldsymbol{x}_i)}||y_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)||^2 + \frac{1}{2}\log\sigma_{\boldsymbol{\theta}}^2(\boldsymbol{x}_i).$$

https://arxiv.org/pdf/2302.13425

Side Note: Multi-task Learning



$$\frac{1}{2\sigma_1^2}\mathcal{L}_1(\mathbf{W}) + \frac{1}{2\sigma_2^2}\mathcal{L}_2(\mathbf{W}) + \log\sigma_1\sigma_2$$

Kendall, A., Gal, Y., & Cipolla, R. (2018). Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 7482-7491).

$$\mathcal{L}_{\beta\text{-VAE}} = -E_{q(\mathrm{z}|\mathrm{x})}\Big[\log \underbrace{p(\mathrm{x}|\mathrm{z})}_{Decoder}\Big]$$

$$+\beta \cdot D_{KL}\Big(\underbrace{q(\mathrm{z}|\mathrm{x})}_{Encoder} \parallel p(\mathrm{z})\Big),$$

$$\mathcal{L}_{\text{L-VAE}} = -\frac{1}{\sigma_0^2}E_{q(\mathrm{z}|\mathrm{x})}\Big[\log p(\mathrm{x}|\mathrm{z})\Big] + \frac{1}{\sigma_1^2}\mathcal{D}_{KL}\Big(q(\mathrm{z}|\mathrm{x}) \parallel \mathrm{z}\Big) + \sum_{i=0,1}\sigma_i^2.$$

H. Mogultay Ozcan, S. Kalkan, F. Y. Vural, "L-VAE: Variational Auto-Encoder with Learnable Beta for Disentangled Representation", Machine Vision and Applications, 36(104):1-14, 2025.

Methods for Estimating Aleatoric Uncertainty

For classification problems:

(passing inputs through the model to get logits). We only need to sample from the logits, which is a fraction of the network's compute, and therefore does not significantly increase the model's test time. We can rewrite the above and obtain the following numerically-stable stochastic loss:

$$\hat{\mathbf{x}}_{i,t} = \mathbf{f}_i^{\mathbf{W}} + \sigma_i^{\mathbf{W}} \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I)$$

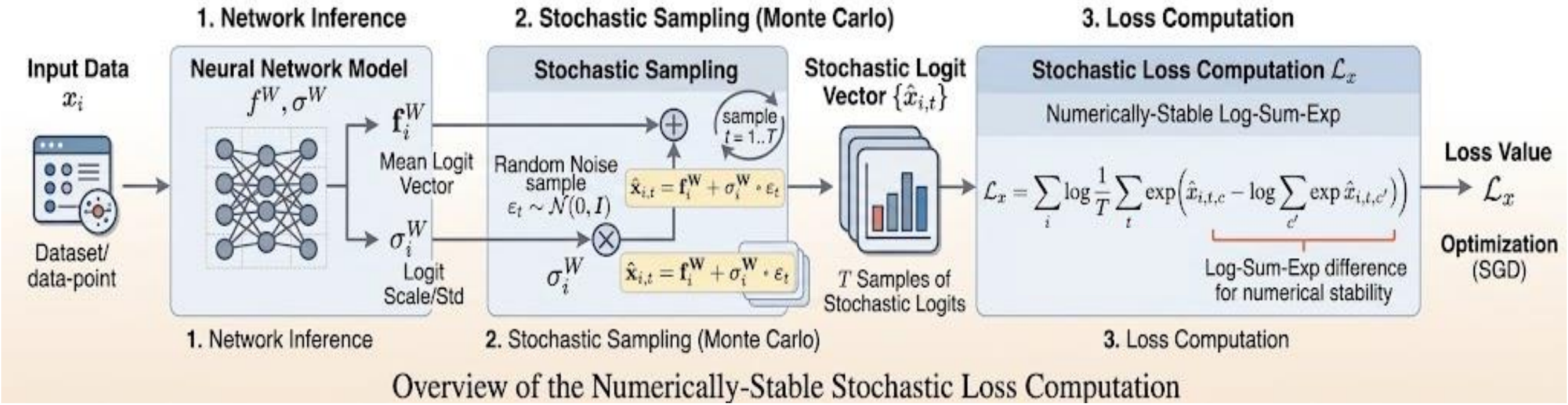
$$\mathcal{L}_x = \sum_i \log \frac{1}{T} \sum_t \exp(\hat{x}_{i,t,c} - \log \sum_{c'} \exp \hat{x}_{i,t,c'}) \tag{12}$$

with $x_{i,t,c'}$ the c' element in the logit vector $\mathbf{x}_{i,t}$.

"What uncertainties do we need for computer vision?"
https://arxiv.org/pdf/1703.04977

Methods for Estimating Aleatoric Uncertainty

For classification problems:



Overview of the Numerically-Stable Stochastic Loss Computation

"What uncertainties do we need for computer vision?"
https://arxiv.org/pdf/1703.04977

Using Uncertainty [from our research]

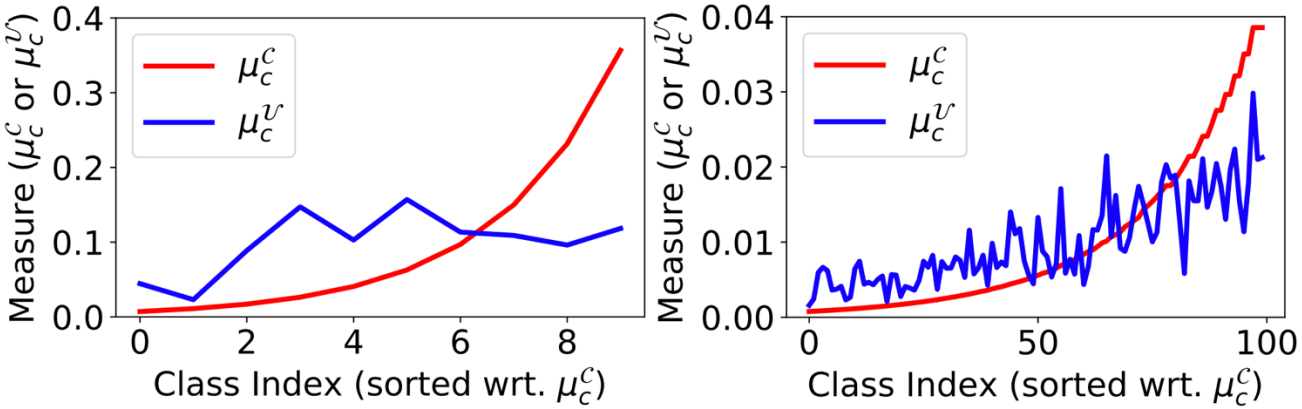
Class Uncertainty:
A Measure to Mitigate Class Imbalance

Zeynep Sonat Baltaci[1,2], Kemal Oksuz[3], Selim Kuzucu[1], Kivanc Tezoren[1], Berkin Kerim Konar[1], Alpay Ozkan[1], Emre Akbas[1,4,†], Sinan Kalkan[1,4,†]
[1]Dept. of Computer Engineering, METU, Ankara, Turkey
[2]LIGM, Ecole des Ponts, Univ Gustave Eiffel, CNRS, Marne-la-Vallée, France
[3]Five AI Ltd., United Kingdom
[4]Center for Robotics and Artificial Intelligence (ROMER), METU, Ankara, Turkey

https://arxiv.org/pdf/2311.14090

- Measuring data imbalance using epistemic and aleatoric uncertainty

- Alternative:
  - Class cardinality
  - Loss values
  - Prediction probability



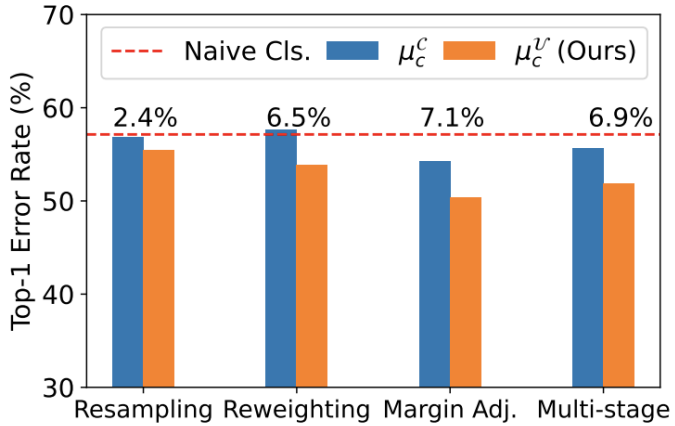
(a) μ^C vs. μ^U on CIFAR-10-LT (b) μ^C vs. μ^U on CIFAR-100-LT

Using Uncertainty [from our research]

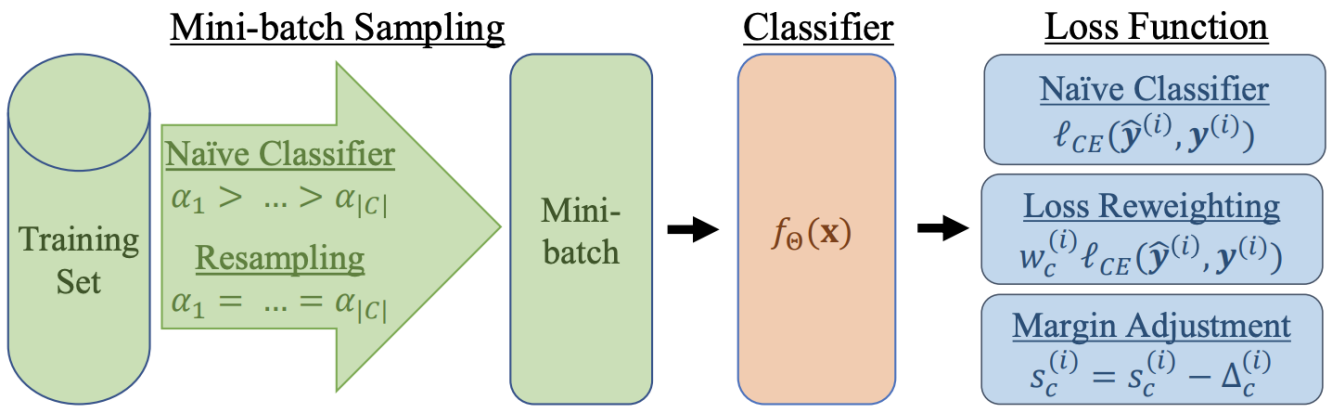
Class Uncertainty:
A Measure to Mitigate Class Imbalance

Zeynep Sonat Baltaci[1,2], Kemal Oksuz[3], Selim Kuzucu[1], Kivanc Tezoren[1], Berkin Kerim Konar[1], Alpay Ozkan[1], Emre Akbas[1,4,†], Sinan Kalkan[1,4,†]
[1]Dept. of Computer Engineering, METU, Ankara, Turkey
[2]LIGM, Ecole des Ponts, Univ Gustave Eiffel, CNRS, Marne-la-Vallée, France
[3]Five AI Ltd., United Kingdom
[4]Center for Robotics and Artificial Intelligence (ROMER), METU, Ankara, Turkey

https://arxiv.org/pdf/2311.14090



(a) Gains of our CLASS UNCERTAINTY



(b) Naïve classifier vs. different ways of handling imbalance

Fig. 1. **(a)** Using our CLASS UNCERTAINTY improves top-1 error rate of all aforementioned methods (CIFAR-100-LT with an IR of 50 using ResNet-32). In particular, we obtain (i) the sampling probability of each class in resampling methods; (ii) the weights of the loss for each class; (iii) the margins to be enforced around each class; or (iv) again sampling probability in the second-stage of multi-stage training strategy using CLASS UNCERTAINTY. The numbers on top of the histograms show relative gain over cardinality-based methods. Baselines: Progressively-balanced sampling [2], class-balanced Focal Loss [3], LDAM with reweighting [7] and deferred resampling. **(b)** Naïve classifier is not robust to class imbalance, giving rise to a plethora of mitigation methods: (1) "resampling" samples a balanced set of examples; (2) "reweighting" assigns a weight for each class ($w_c^{(i)}$); (3) "margin-adjustment" methods assign different margins ($\Delta_c^{(i)}$) to the logits ($s_c^{(i)}$) of different classes; and (4) "multi-stage training" first trains a naïve classifiers followed by a resampling or reweighting method to re-train or fine-tune the classifier. These methods generally rely on the cardinality of the classes.

Using Uncertainty [from our research]

Journal of Artificial Intelligence Research 80 (2024) 12-15 Submitted 02/2024; published 08/2024

Uncertainty as a Fairness Measure

Selim Kuzucu SELIM.KUZUCU@METU.EDU.TR
Department of Computer Engineering
Middle East Technical University
06800 Ankara, Turkiye

Jiaee Cheong JC2208@CAM.AC.UK
Department of Computer Science
University of Cambridge
Cambridge, CB3 0FD, United Kingdom
The Alan Turing Institute
London, NW1 2DB, United Kingdom

Hatice Gunes HG410@CAM.AC.UK
Department of Computer Science
University of Cambridge
Cambridge, CB3 0FD, United Kingdom

Sinan Kalkan SKALKAN@METU.EDU.TR
Department of Computer Engineering &
ROMER Robotics-AI Center
Middle East Technical University
06800 Ankara, Turkiye

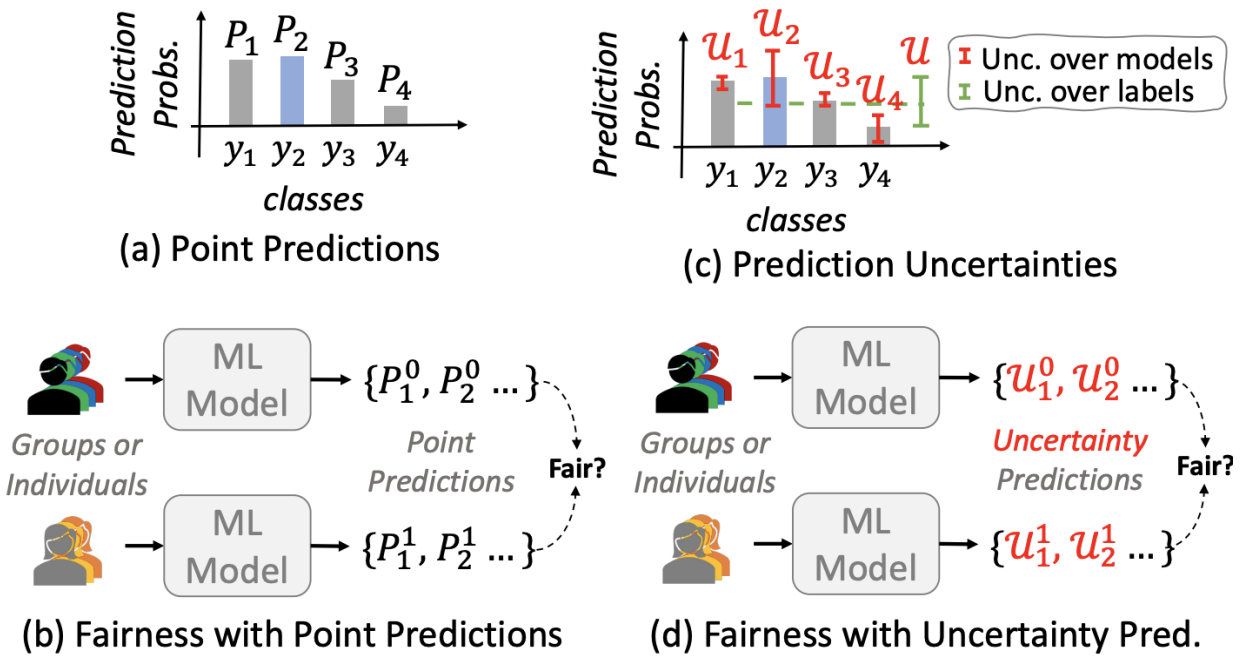


Figure 1: Existing fairness measures utilize point predictions for quantifying fairness, which ignores the uncertainty (variance) of the predictions **(a-b)**. We fill this gap by using uncertainty instead for measuring fairness **(c-d)**.